

The Google File System

by S. Ghemawat, H. Gobioff, and S-T. Leung

CSCI 585 discussion by Shahram Ghandeharizadeh
Computer Science Department
University of Southern California

Primary Functionality of Google



Primary Functionality of Google

- Search content on the web in browsing mode.
- Open world assumption: If your search with Google does not return results, it does not mean that the referenced content is non-exist. It only means that Google does not know about it when the search was issued.
 - Google may retrieve results if the search is issued again.
 - Do not index/find me: Google provides tags to enable an information provider to prevent Google from indexing its pages.
 - No one gets angry if Google does not retrieve information known to exist on the Internet.

**How is this different
than financial
applications?**

Functionality

IR: 

- Search content on the web in browsing mode.
- Open world assumption: If your search with Google does not return results, it does not mean that the referenced content is non-existent. It only means that Google does not know about it when the search was issued.
 - Google may retrieve results if the search is issued again.
 - Do not index/find me: Google provides tags to enable an information provider to prevent Google from indexing its pages.
 - No one gets angry if Google does not retrieve information known to exist on the Internet.

DB:



- Query based: Looking for a needle in a hay stack.
- Closed world assumption: A data item that is not known does not exist.
 - A query must retrieve correct results 100% of the time!
 - If a customer insists the bank cannot find his or her account because the customer has used Google's "do not find me" tags, the customer is kicked out!
- Customers become angry if the system retrieves incorrect data.



Key Observation

IR: 

- Okay to return either no or incorrect results.
- Acceptable for a user search to observe stale data.
- Rank ordered list of possible results.

DB:



- **Not** okay to return incorrect results.
- A transaction must observe **consistent** data.
- SQL front end.

Big Picture

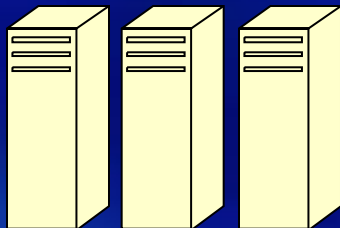
- A shared-nothing architecture consisting of thousands of nodes!
 - A node is an off-the-shelf, commodity PC.



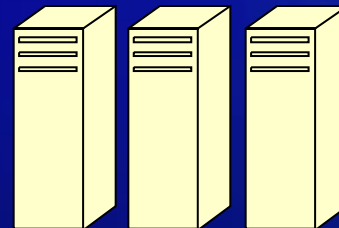
Map/Reduce

Bigtable Data Model

Google File System

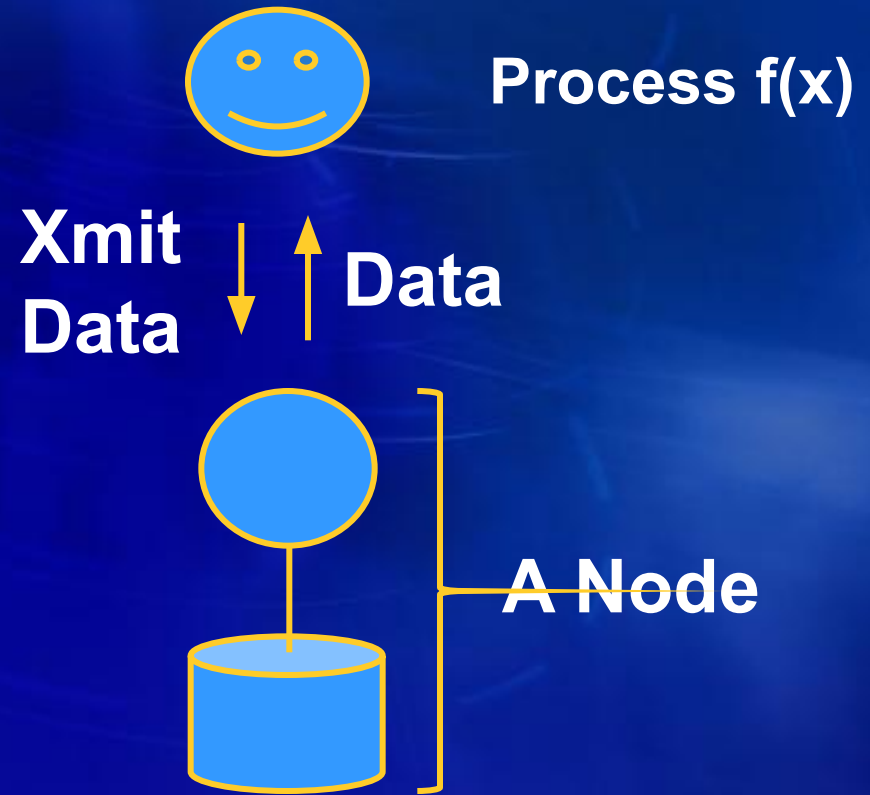


.....



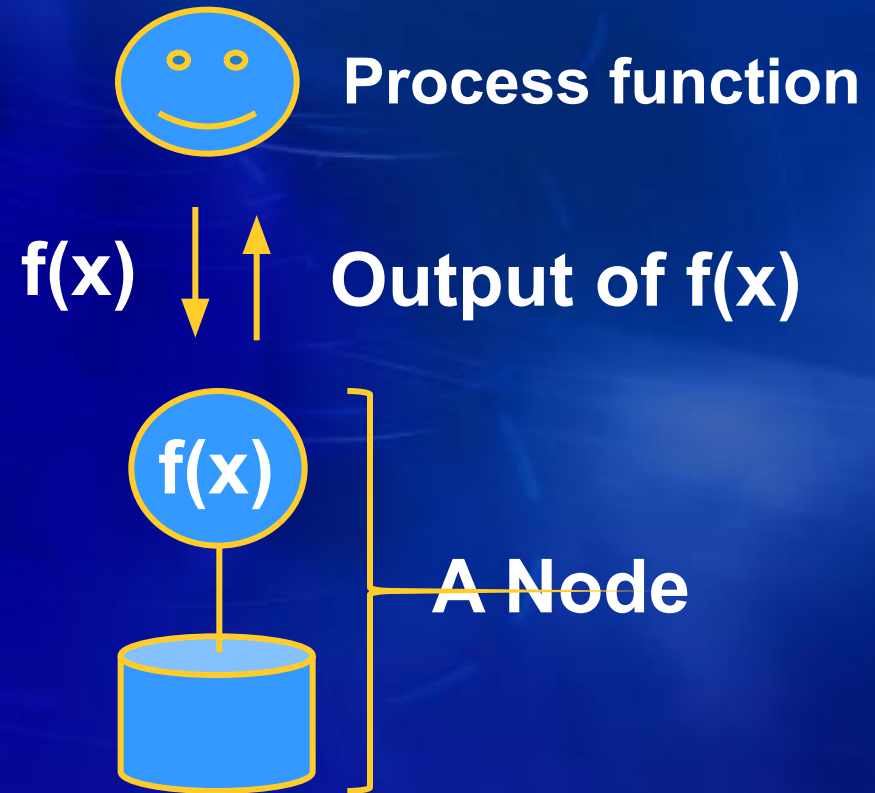
Data Shipping

- Client retrieves data from the node.
- Client performs computation locally.
- Limitation: Dumb servers, utilizes the limited network bandwidth.



Function Shipping

- Client ships the function to the node for processing.
- Relevant data is sent to client.
- Function $f(x)$ should produce less data than the original data stored in the database.
- Minimizes demand for the network bandwidth.



Google

- **Application (configured with GFS client) may run on the same PC as the one hosting a chunkserver. Requirements:**
 - ▣ **Machine resources are not overwhelmed.**
 - ▣ **The lower reliability is acceptable.**

References

- **Map Reduce**

- ▣ Dean and Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM, Vol. 51, No. 1, January 2008.

- **Bigtable**

- ▣ Chang et. al. Bigtable: A Distributed Storage System for Structured Data. In OSDI 2006.

- **GFS**

- ▣ Ghemawat et. al. The Google File System. In SOSP 2003.

Overview: GFS

- **A highly available, distributed file system for inexpensive commodity PCs.**
 - **Supports node failures as the norm rather than the exception.**
 - **Stores and retrieves multi-GB files.**
 - **Assumes files are append only (instead of updates that modify a certain piece of existing data).**
 - **Atomic append operation to enable multiple clients to append to a file with minimal synchronization.**
 - **Relaxed consistency model to simplify the file system and enhance performance.**

Google File System: Assumptions

- The system is built from many inexpensive commodity components that often fail. It must constantly monitor itself and detect, tolerate, and recover promptly from component failures on a routine basis.
- The system stores a modest number of large files. We expect a few million files, each typically 100 MB or larger in size. Multi-GB files are the common case and should be managed efficiently. Small files must be supported, but we need not optimize for them.
- The workloads primarily consist of two kinds of reads: large streaming reads and small random reads. In large streaming reads, individual operations typically read hundreds of KBs, more commonly 1 MB or more. Successive operations from the same client often read through a contiguous region of a file. A small random read typically reads a few KBs at some arbitrary offset. Performance-conscious applications often batch and sort their small reads to advance steadily through the file rather than go back and forth.

Google File System: Assumptions (Cont...)

- The workloads also have many large, sequential writes that append data to files. Typical operation sizes are similar to those for reads. Once written, files are seldom modified again. Small writes at arbitrary positions in a file are supported but do not have to be efficient.
- The system must efficiently implement well-defined semantics for multiple clients that concurrently append to the same file. Our files are often used as producer-consumer queues or for many-way merging. Hundreds of producers, running one per machine, will concurrently append to a file. Atomicity with minimal synchronization overhead is essential. The file may be read later, or a consumer may be reading through the file simultaneously.
- High sustained bandwidth is more important than low latency. Most of our target applications place a premium on processing data in bulk at a high rate, while few have stringent response time requirements for an individual read or write.

GFS: Interfaces

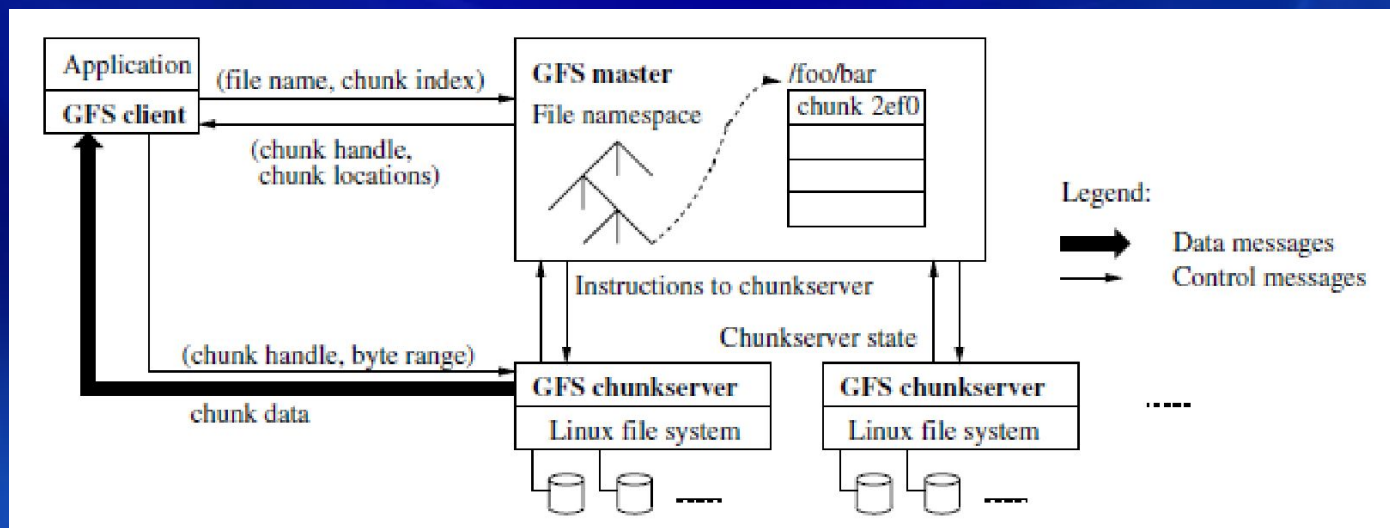
- Create, delete, open, close, read, and write files.
- Snapshot a file:
 - Create a copy of the file.
- Record append operation:
 - Allows multiple clients to append data to the same file concurrently, while guaranteeing the atomicity of each individual client's append.

GFS: Architecture

- 1 Master
- Multiple chunkservers
- File is partitioned into fixed-size chunks.
- Each chunk has a 64 bit chunk handle that is unique globally.
- Each chunk is replicated on several chunkservers.
 - Degree of replication is application specific; default is 3.

- Software

- Master maintains all file system meta-data: namespace, access control info, mapping from files to chunks, current location of chunks.
- GFS client caches meta-data about file system.
- Client receives data from chunkserver directly.
- Client and chunkserver do not cache file data.

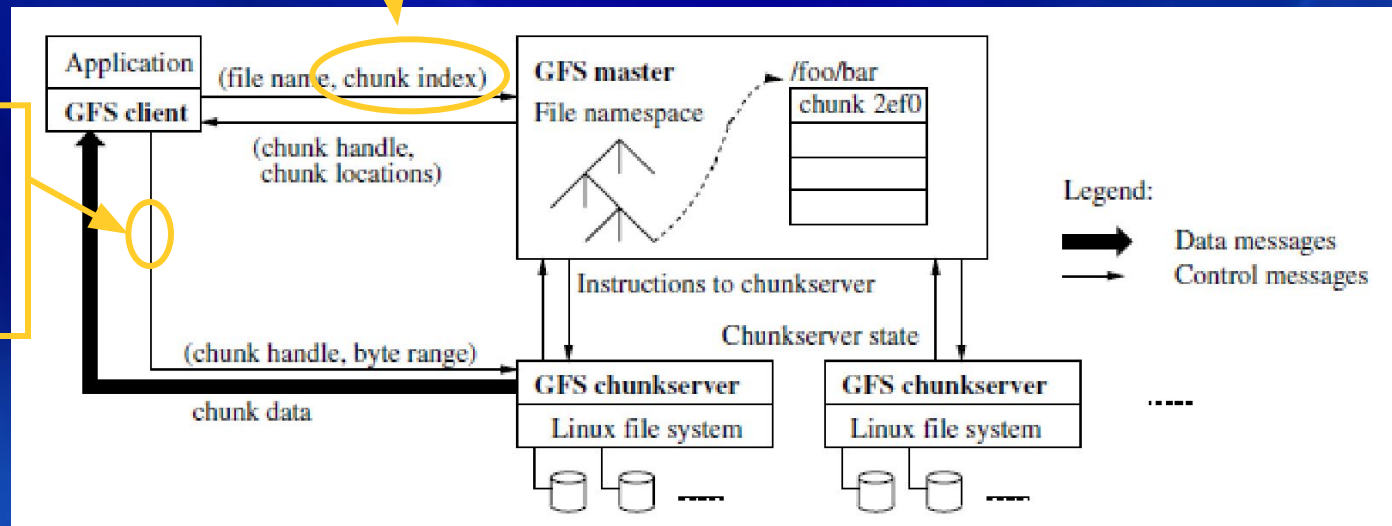


GFS: Architecture

- 1 Master
- Multiple chunkservers
- File is partitioned into fixed-size (64 MB) chunks.
- Each chunk has a 64 bit chunk handle that is unique globally.
- Each chunk is replicated on several chunkservers.
 - Degree of replication is application specific; default is 3.

- Software

- Master maintains all file system meta-data: namespace, access control info, mapping from files to chunks, current location of chunks.
- GFS client caches meta-data about file system.
- Client receives data from chunkserver directly.
- Client and chunkserver do not cache file data.



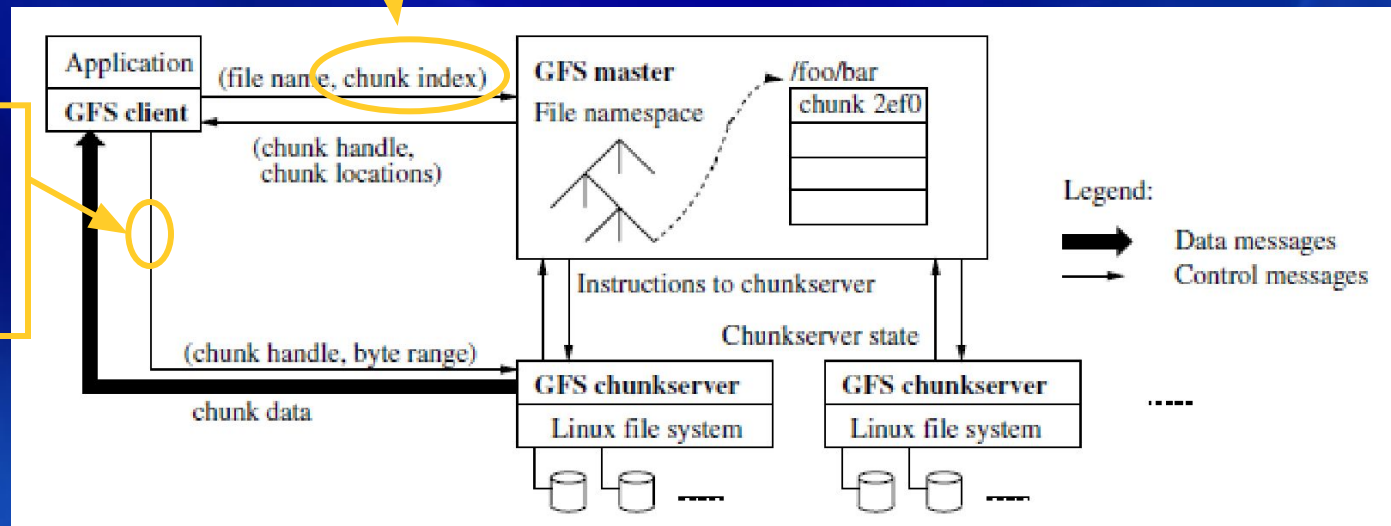
GFS: Architecture

- Unix allocates space lazily!
- Many small logical files are stored in one file.

- 1 Master
- Multiple chunkservers
- File is partitioned into fixed-size (64 MB) chunks.
- Each chunk has a 64 bit chunk handle that is unique globally.
- Each chunk is replicated on several chunkservers.
 - Degree of replication is application specific; default is 3.

Software

- Master maintains all file system meta-data: namespace, access control info, mapping from files to chunks, current location of chunks.
- GFS client caches meta-data about file system.
- Client receives data from chunkserver directly.
- Client and chunkserver do not cache file data.



GFS Master

- 1 master simplifies software design.
- Master monitors availability of chunkservers using heart-beat messages.
- 1 master is a single point of failure:
 - Master does not store chunk location information persistently: When the master is started, it asks each chunkserver about its chunks (and whenever a chunkserver joins).
 - File and chunk namespaces,
 - Mapping from files to chunks,
 - Location of each chunk's replica.